

graphify

Repository Analysis Report

A knowledge graph builder for code repositories used with AI coding assistants.

■ Files	25	■ Lines	13,122
■ Architecture	Library/CLI Tool	■ Provider	anthropic/claude-sonnet-4-20250514
■ Date	April 09, 2026	■ Languages	Python (18), Markdown (7)

Table of Contents

1.	Executive Summary
2.	Technology Stack
3.	Architecture Overview
4.	System Architecture Diagram
5.	Data Flow Diagram
6.	Component Diagram
7.	Key Components
8.	API Endpoints
9.	Code Quality & Security
10.	File-by-File Analysis
11.	Improvement Recommendations
12.	Repository File Tree

1. Executive Summary

Graphify builds semantic knowledge graphs from code repositories. It extracts AST nodes and relationships, clusters them into communities using Leiden algorithm, and exposes a BFS/DFS traversal API for LLM context retrieval. The tool integrates with Claude Code, Codex, OpenCode, and other AI assistants via skill files.

Design Patterns

- Strategy Pattern
- Builder Pattern
- Observer (watch.py)

Testing Strategy

Unit tests per module with benchmark tooling

Deployment

Installed as Python package via pip, used as CLI or library

Entry Points

- graphify/__main__.py

2. Technology Stack

Technologies & Frameworks

Python	NetworkX	tree-sitter	FastAPI
Leiden Algorithm			

External Dependencies

- networkx
- tree-sitter
- graspologic
- scipy

Language Distribution

Language	Files	Share
Python	18	72.0%
Markdown	7	28.0%

3. Architecture Overview

Architecture Style: **Library/CLI Tool**

Data Flow

1. User runs /graphify command
2. Files scanned and parsed by extract.py
3. Nodes and edges built in build.py
4. Communities detected in cluster.py
5. GRAPH_REPORT.md written
6. LLM queries graph via serve.py BFS/DFS

4. System Architecture Diagram

High-level system components and their relationships.

[Architecture - diagram code]

```
graph TD
  CLI[CLI __main__.py] --> Build[build.py]
  Build --> Extract[extract.py]
  Build --> Cluster[cluster.py]
  Build --> Analyze[analyze.py]
  Analyze --> Report[report.py]
  CLI --> Serve[serve.py]
  Serve --> Graph[(graph.json)]
```

5. Data Flow Diagram

Main request / data flow through the system.

[Data Flow - diagram code]

```
sequenceDiagram
participant User
participant CLI
participant Build
participant Extract
participant Cluster
participant Report
User->>CLI: graphify /path
CLI->>Build: build_graph(path)
Build->>Extract: parse files
Extract-->>Build: nodes + edges
Build->>Cluster: detect communities
Cluster-->>Build: community map
Build->>Report: generate GRAPH_REPORT.md
Report-->>User: graphify-out/
```

6. Component Diagram

Module dependencies and internal structure.

[Components - diagram code]

```
graph LR
  subgraph Core
    B[build.py]
    E[extract.py]
    C[cluster.py]
  end
  subgraph Analysis
    A[analyze.py]
    R[report.py]
    S[serve.py]
  end
  subgraph IO
    I[ingest.py]
    W[wiki.py]
    X[export.py]
  end
  Core --> Analysis
  Analysis --> IO
```

7. Key Components

Graph Builder

Builds knowledge graph from extracted AST nodes

- Node creation
- Edge inference

Files: `build.py`

Depends on: **Extract**, **Cluster**

AST Extractor

Parses source code into nodes and edges using tree-sitter

- Parse Python/TS/Go/Rust
- Extract classes, functions, imports

Files: `extract.py`

Cluster

Groups nodes into semantic communities using Leiden algorithm

- Community detection
- Cohesion scoring

Files: `cluster.py`

Depends on: **Build**

9. Code Quality & Security

Code Quality Observations

- Well-structured modular design
- Good separation of concerns
- Type hints throughout

Security Considerations

- URL validation in security.py
- Safe fetch wrappers prevent SSRF

10. File-by-File Analysis

`graphify/analyze.py`

MEDIUM

Finds god nodes (most connected) and surprising cross-community connections

Functions: `god_nodes`, `surprising_connections`, `suggest_questions`

Core analysis logic

`graphify/build.py`

HIGH

Orchestrates the full graph building pipeline

Functions: `build_graph`

Main entry point for graph construction

`graphify/extract.py`

HIGH

AST extraction using tree-sitter for multiple languages

Classes: `LanguageConfig`

Functions: `extract_file`

Supports Python, TS, Go, Rust, Java, C/C++

11. Improvement Recommendations

1. Add async support for large repos
2. Add caching layer for LLM calls
3. Add GitHub Actions CI

12. Repository File Tree

[illegible]